

EAG Four-Role RAG Agent — Architecture & Features

Extensive AI Agents (EAG V3 Session 7) · Memory · Perception · Decision · Action · FAISS vector memory (RAG) · 12 MCP tools

1. Executive summary

This project implements a **cognitive agent** that answers multi-step questions without a monolithic “do everything” prompt. Work is split across four roles—**Memory**, **Perception**, **Decision**, and **Action**—each with a narrow contract (Pydantic models). The orchestrator (`agent7.py`) runs a loop until Perception marks all goals complete.

Session 7 extensions:

- **Vector memory (RAG):** FAISS index over embeddings from LLM Gateway V7 (`/v1/embed` , Ollama `nomic-embed-text`).
- **Document indexing:** Chunk workspace files and corpus items into searchable facts.
- **Part 2:** Custom RAG over 1,500 AI job postings (Kaggle dataset) in an isolated `state_jobs/` store.

All LLM calls go through the course **LLM Gateway** (port 8107 preferred). MCP tools run in a separate stdio server process.

2. Design principles

Principle	How it is enforced
Separation of concerns	Each role has one job; schemas define what crosses boundaries.
One goal per Decision step	Decision sees only the current goal text, not the full plan.
Intent vs tools	Perception plans in natural-language intent (fetch, query corpus, compare). Decision maps intent to MCP tools via tool docstrings + <code>decision_system.txt</code> .
Rendering-first debugging	Memory hits and history expose <code>chunk:</code> , <code>raw:</code> , <code>search-results:</code> text to prompts before adding new SYSTEM rules.
Durable state	<code>state/</code> (course) and <code>state_jobs/</code> (Part 2) persist memory + FAISS across runs.
Large payloads as artifacts	Pages > 4 KB → <code>art:...</code> store; Memory keeps handles; Perception attaches when needed.

3. System architecture

3.1 High-level flow



3.2 Process layout

Component	File	Role
Orchestrator	<code>agent7.py</code>	Loop, CLI presets, MCP session, state dir selection
Memory	<code>memory.py</code>	JSON store + FAISS; vector then keyword retrieval
Perception	<code>perception.py</code>	Goal decomposition, done flags, artifact attachment
Decision	<code>decision.py</code>	One LLM call: plain answer or single tool call
Action	<code>action.py</code>	Dispatches MCP <code>call_tool</code>
MCP server	<code>mcp_server.py</code>	12 tools (stdio)
Gateway client	<code>gateway.py</code>	Chat + embed via V7/V2
Artifacts	<code>artifacts.py</code>	Content-addressed blobs under <code>state/artifacts/</code>
Vector index	<code>vector_index.py</code>	FAISS <code>IndexFlatIP</code> + <code>index.faiss</code> / <code>index_ids.json</code>
Indexing	<code>indexing.py</code>	Shared chunking → <code>Memory.add_fact</code>
Prompt rendering	<code>prompt_render.py</code>	Formats hits for Perception/Decision
History	<code>history_utils.py</code>	Final answer extraction, action summaries

3.3 MCP integration

The agent spawns `mcp_server.py` as a subprocess and passes `EAG_STATE_DIR` when using Part 2 presets (`j1 - j5`). Tools read/write under `workspace/` and index into the active Memory path.

4. The four roles (detailed)

4.1 Memory

Responsibilities:

- **remember** — Classify and store the user query (and similar inputs).
- **read** — Retrieve top-k items for the current query + run history (vector search first, keyword fallback).
- **record_outcome** — Store tool results with embeddings and optional artifact IDs.
- **add_fact** — Used by indexing (`index_document`, `index_directory`, bulk jobs index).

Storage:

- `memory.json` — List of `MemoryItem` records.
- `index.faiss` + `index_ids.json` — Embedding index (768-dim, L2-normalized inner product).

Item kinds: `fact`, `preference`, `tool_outcome`, `scratchpad`.

Retrieval priority: Indexed chunks (`[workspace:...]`, `[corpus:...]`, `chunk:` in value) rank above query-echo scratchpad when vector hits mix both.

4.2 Perception

Responsibilities:

- Decompose the user query into an ordered **goal list** (minimum goals needed).
- Mark `done=true` only when **run history** shows a satisfying outcome (not from the LLM's done flag alone).
- Set `attach_artifact_id` for synthesis/compare goals when a prior fetch artifact exists.
- **Never name MCP tools** in goals (architectural gate).

Seeded goals (code):

- URL queries → Fetch + Extract (preset **a** pattern).
- Indexed papers already in memory → RAG + compare/synthesise (presets **f2**, **h** pattern).

LLM: Gemini via gateway (`provider=g`), structured JSON (`PerceptionLLMOutput`).

4.3 Decision

Responsibilities:

- See **one** goal, memory hits, attached artifacts, recent history, and MCP tool schemas.
- Return **exactly one** of: substantive plain-text answer **or** a single tool call.
- Policy in `prompts/decision_system.txt`; runtime hints in `RUN CONTEXT` (e.g. URLs already fetched, enough search results to answer).

LLM routing: `auto_route=decision`; large attachments force Gemini without tools.

4.4 Action

Responsibilities:

- Execute the tool via MCP.
- Return text + optional artifact ID for large responses.
- No LLM in Action.

5. MCP tools (12)

Tool	Purpose
web_search	Tavily primary, DuckDuckGo fallback (capped results)
fetch_url	HTTP fetch; Wikipedia via MediaWiki API
get_time	Time in a named timezone
currency_convert	FX conversion
read_file	Read under workspace/
list_dir	List under workspace/
create_file	Create under workspace/ (reminders, notes)
update_file	Overwrite under workspace/
edit_file	Search/replace under workspace/
index_document	Chunk + embed one file into Memory
index_directory	Index all *.md under a folder (preset f1)
search_knowledge	Vector search over indexed chunks

6. Session 7 features (RAG & indexing)

6.1 Chunking and indexing

- Text split into overlapping word chunks (~400 words, 80 overlap).
- Each chunk stored as a `fact` with descriptor `[source chunk i/n] preview...`.
- Embeddings via gateway `retrieval_document` task type.

6.2 Course papers (presets e-h)

- Source files in `papers/` → copied to `workspace/papers/`.
- **e**: index one paper, answer from corpus.
- **f1**: `index_directory` ON `papers/`.
- **f2 / g / h**: `search_knowledge` + synthesise/compare (keep `state/` between **f1** and **f2**).

6.3 Part 2 — AI Jobs Market RAG

Stage	Output
Kaggle CSV	<code>data/ai_jobs/raw/</code> (gitignored)
Corpus build	<code>corpus/ai_jobs/items/*.md</code> (1500 items)
Bulk index	<code>state_jobs/</code> (isolated FAISS + memory)
CLI	<code>jobs_rag.py</code> presets q1-q5
Agent	<code>agent7.py</code> presets j1-j5 (same queries, <code>EAG_STATE_DIR=state_jobs</code>)

Custom query types: semantic (q1-q3), factual (q4), comparison (q5).

7. Query presets

7.1 Part 1 (course)

Preset	Capability tested
a	<code>fetch_url</code> + artifact attach + extract
b	<code>web_search</code> + weather + multi-goal synthesis
c1	<code>create_file</code> reminders + memory
c2	Recall from prior run (<code>state/</code> retained)
d	Search + read + numbered synthesis
e	<code>index_document</code> + RAG answer
f1	<code>index_directory</code> bulk ingest
f2	Cross-corpus semantic query
g	Semantic recall (credit assignment)
h	Cross-paper comparison (ReAct vs CoT)

7.2 Part 2 (AI jobs)

Preset	Maps to	Type
j1 / q1	IC modeling roles, strong pay	semantic
j2 / q2	Remote ML > \$200k	semantic
j3 / q3	Early career outside US	semantic
j4 / q4	Max NLP/LLM compensation	factual
j5 / q5	UK AI engineer vs data scientist pay	comparison

8. Data contracts (`schemas.py`)

- **MemoryItem** — Stored memory unit with optional embedding and artifact link.
- **Goal** — `id`, `text`, `done`, `attach_artifact_id`.
- **Observation** — List of goals; `all_done`, `next_unfinished()`.
- **DecisionOutput** — XOR: `answer` or `tool_call`.
- **PerceptionLLMOutput** — Goals with optional `artifact_index` into formatted memory hits.

9. Runtime directories

Path	Committed?	Contents
state/	No	Course agent memory + FAISS
state_jobs/	No	Part 2 jobs index (~tens of MB)
workspace/	No	Agent file I/O, copied papers/jobs
logs/	No	Run logs from scripts
corpus/ai_jobs/	Yes (items + manifest)	Built markdown corpus
papers/	Yes	Course paper sources

10. External dependencies

1. **Python 3.11+** and **uv** (`uv sync`).
2. **LLM Gateway V7** on `http://localhost:8107` (chat + `/v1/embed`).
3. **Ollama** with `nomic-embed-text` for embeddings.
4. **API keys** in `.env` (`GEMINI_API_KEY` , optional `TAVILY_API_KEY`).

11. Testing & quality gates

```
uv run pytest                # 26+ unit tests (no gateway required)
./scripts/check-perception-gate.sh  # No MCP tool names in perception_system.txt
```

Tests cover memory retrieval, perception history/done rules, indexed corpus seeds, prompt gates, and corpus smoke checks.

12. Typical workflows

Single preset:

```
./scripts/run.sh python agent7.py --preset a
```

Part 2 after indexing:

```
./scripts/run_custom_rag_traces.sh
./scripts/run_custom_queries.sh
```

Regenerate architecture PDF:

```
./scripts/build_architecture_pdf.sh
```